

Part D: AI Usage Report

DSC 190/291 — Assignment 1

Student: Zeyu Bian

Which parts used AI?

I used AI assistance for all four parts of this assignment:

- **Part A (repo setup):** Asked AI to help draft the `CLAUDE.md` workflow file, including the section describing how it should assist with proofs and experiments.
 - **Part B (theory):** AI helped sketch the initial proof structure for B.1 and B.2. I verified each step independently and revised the wording to match what was covered in lecture.
 - **Part C (implementation):** AI wrote the first draft of `perceptron.py`, including the data generation function and the experiment loops. I reviewed the code, identified a flaw in the initial data generation procedure (explained below), and added the proof-invariant sanity checks.
 - **Part D (this report):** Drafted by me, with AI suggesting structure.
-

A suggestion I accepted

In the proof of B.2, I initially tried to use the feature map $\phi(x) = x$ (just the scalar x) with some scaling. AI suggested the 2D map $\phi(x) = (x, 1)$, which naturally embeds the threshold function $h_{\theta}(x) = \text{sign}(x - \theta)$ as a linear classifier via $w^* = (1, -\theta)$. I accepted this immediately because it is the standard trick for turning affine classifiers into linear ones (the "bias absorbed into the weight vector" idea), and it made the margin calculation clean and tight.

A suggestion I rejected / modified

The first draft of `generate_data` drew the first coordinate as $|x_1| \sim \text{Uniform}[\gamma, R]$. This is valid (the margin condition $|x_1| \geq \gamma$ holds), but it causes the effective margin to be approximately $(\gamma + R)/2$, much larger than γ for small γ . When I plotted the log-log slope of mistakes vs. $1/\gamma^2$, the slope came out to 0.71 instead of the expected ~ 1.0 , because the easy examples at large margin dominated and suppressed mistake counts at small γ .

I modified the data generator to support a `tight_margin` flag that draws $|x_1|$ from a narrow band $[\gamma, \gamma + \epsilon]$ (with $\epsilon = 0.02$), keeping the effective margin close to γ . This improved the log-log slope to 0.84. The remaining gap from 1.0 is expected: the theoretical bound is worst-case, while our data uses a random ordering; on random sequences, Perceptron typically makes $\sim 3x$ fewer mistakes than the bound predicts (e.g., ~ 36 vs. 100 at $\gamma = 0.1$).

How I verified correctness

1. **Proof invariants.** The Perceptron convergence proof relies on two invariants after M mistakes: $w_M \cdot u^* \geq M/\gamma$ and $|w_M|^2 \leq MR^2$. I added

`verify_perceptron_bounds()` to check both numerically on every run. Both passed on all test cases.

2. **Sign convention at $w=0$.** The convention `sign(0) = +1` affects which examples are mistakes at initialization. I added a sanity check (Check 1 in the code) that traces through a one-example prediction by hand and asserts the correct number of updates.
3. **Data generation checks.** `verify_data()` asserts that every generated example satisfies $|x| = R$ (to floating-point precision) and $y \cdot (u^* \cdot x) \geq \gamma$, catching any bug in the normalization step.
4. **Determinism.** Check 4 confirms that identical seeds produce identical datasets, ruling out accidental state-sharing between runs.